VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMATIKOS INSTITUTAS
INFORMATIKOS KATEDRA

Realios ir virtualios mašinos aprašas

Atliko: Informatikos 3 kurso 1 grupės studentai

Ainis Augustas Laurinavičius, Faustas Avulis, Šarūnas Sinkevičius (parašas)

Darbo vadovas:

Mantas Grubliauskis (parašas)

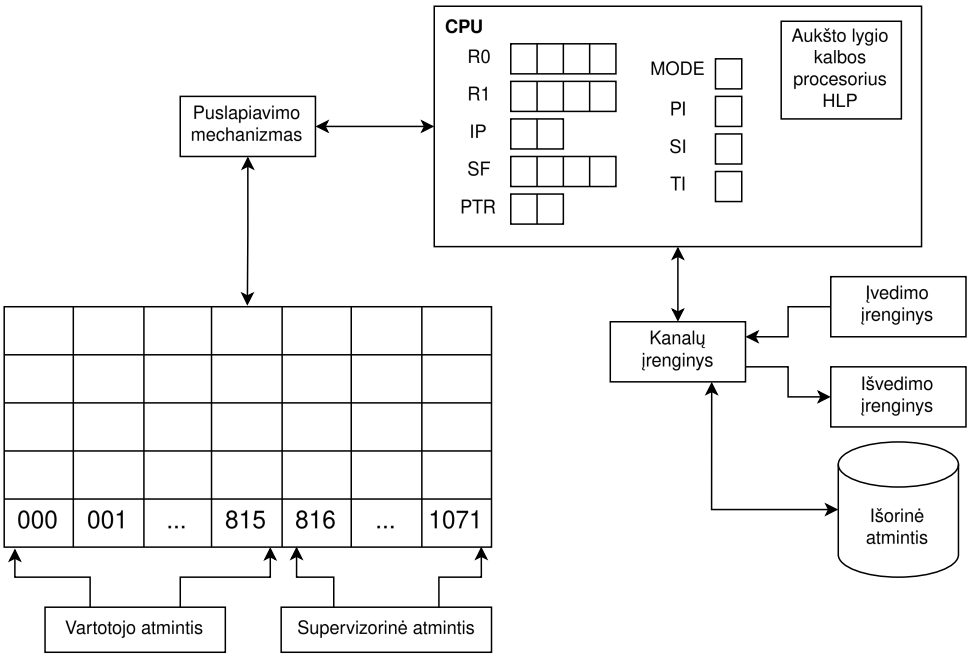
Vilnius
2026

Turinys

1. Realios mašinos aprašas 2
   1.1. Procesorius 2
   1.2. Atmintis 3
   1.3. Kanalų įrenginys 3
   1.4. Įvedimo/išvedimo įrenginiai 4
2. Virtualios mašinos aprašas 5
   2.1. Virtualios mašinos samprata 5
   2.2. Procesorius 5
   2.3. Procesoriaus komandos 6
   2.4. Atmintis 7
   2.5. Puslapiavimas 7
   2.6. Pertraukimai 8
   2.7. Įvedimas ir išvedimas 9
   2.8. Vykdomieji failai 9
      2.8.1. Pradžios žymė 9
      2.8.2. Programa 9
      2.8.3. Pabaigos žymė 9

1. Realios mašinos aprašas

Realią mašiną sudaro procesorius, vartotojo atmintis, supervizorinė atmintis, išorinė atmintis, duomenų perdavimo kanalai, įvedimo/išvedimo įrenginiai:



1 pav. Realios mašinos schema

1.1. Procesorius

Procesoriaus paskirtis - skaityti komandą iš atminties ir ją vykdyti (interpretuoti). Procesorius gali dirbti dviem režimais – supervizoriaus arba vartotojo. Supervizoriaus režime komandos iš supervizorinės atminties yra apdorojamos betarpiškai aukšto lygio kalbos procesoriaus HLP. HLP – bet kuris aukšto lygio kalbos procesorius. Vartotojo režime HLP vykdo užduoties programą - imituoja virtualios mašinos procesorių.

Procesoriaus registrai:

- `PTR` – 2 baitų puslapių lentelės registras.
- `R0` - 4 baitų bendro naudojimo registras.
- `R1` - 4 baitų bendro naudojimo registras.
- `IP` – 2 baitų virtualios mašinos programos skaitiklis.
- `SF` – 4 baitų registras, kuriame po 1 baitą skirta keturioms žymėms: „zero flag“ (`ZF = 1`, kai operacijos rezultatas lygus nuliui, `ZF = 0` kitu atveju), „sign flag“ (`SF = 1`, kai operacijos rezultatas neigiamas, `SF = 0` kitu atveju), „interrupt flag“ (`IF = 1`, kai įvyksta pertraukimas iš išorinių įrenginių, `IF = 0` kitu atveju), „overflow flag“ (`OF = 1`, kai operacijos suma ar skirtumas išeina iš leistino intervalo rėžių ($[0000, FFFF]$ teigiamiem skaičiam ir $[-FFF, -001]$ neigiamiem skaičiam), `OF = 0` kitu atveju).

- `MODE` – 1 baito registras, kurio reikšmė nusako procesoriaus darbo režimą.(vartotojas - 0, supervizorius - 1).
- `PI` – 1 baito programinių pertraukimų registras.
- `SI` – 1 baito supervizorinių pertraukimų registras.
- `TI` – 1 baito taimerio registras.

Taimerio mechanizmas atsakingas už geresnį užduočių išlygiagretinimą. Laikoma, kad ta pati užduotis negali būti vykdoma ilgiau negu N laiko momentų, t.y. vykdoma ne daugiau kaip N einamosios užduoties taktų. Laikysime, kad visos instrukcijos įvykdomos per 1 taktą. Taimerio veikimo principas: pradedant vykdyti virtualios mašinos užduotį `TI` registro reikšmė nustatoma tam tikrai reikšmei, pvz. $N = 10$. Įvykdžius instrukciją, `TI` reikšmė mažinama priklausomai nuo to, per kiek taktų ši instrukcija yra atliekama. Kai `TI` registro reikšmė pasiekia nulį, mikrokomanda `test()` aptinka taimerio pertraukimą.

Procesoriaus darbo režimas nustatomas `MODE` registro reikšme. Kai `MODE = 0`, procesorius veikia vartotojo režimu ir vykdo konkrečios virtualiosios mašinos užduoties programą. Kai `MODE = 1`, procesorius veikia supervizoriaus režimu ir vykdo supervizorinės programos komandas: valdo pertraukimus, įvedimo/išvedimo įrenginius.

1.2. Atmintis

Vienas žodis yra 4 baitai. Vartotojo atminčiai skirta 51 blokas (puslapiai) po 16 žodžių (t.y. žodžiai, atmintyje sunumeruoti nuo 0 iki 815 imtinai). Supervizorinei atminčiai skirta 16 blokų po 16 žodžių (t.y. žodžiai, atmintyje sunumeruoti nuo 816 iki 1071 imtinai). Supervizorinė atmintis yra skirta pačios operacinės sistemos poreikiams, tačiau dėl paprastumo supervizorinėje atmintyje turėjusiais būti komandas ir resursus valdys aukšto lygio kalbos procesorius HLP. Darbą su išorine atmintimi atliks HLP. Procesorius darbą su atmintimis valdo naudodamas kanalų įrenginį.

1.3. Kanalų įrenginys

Kanalų įrenginys leidžia dirbti su atmintimis. Priklausomai nuo nustatytų registrų kanalų įrenginys gali vykdyti apsikeitimą duomenimis visomis galimomis kryptimis. Veiksmai su kanalų įrenginiu atliekami tik supervizoriaus režime. Kanalų įrenginio vartotojo sąsaja:

Kanalų įrenginio registrai:

- SB: Bloko, iš kurio kopijuosime, numeris.
- DB: Bloko, į kurį kopijuosime, numeris.
- FN: Failo pavadinimas.
- FF: Baitų poslinkis bloke (nuo bloko pradžios).
- BL: Perduodamo srauto ilgis baitais.
- R2: 4 baitų bendro naudojimo registras.
- ST: Objekto, iš kurio kopijuosime, numeris:

1. Vartotojo atmintis;
2. Supervizorinė atmintis;
3. Išorinė atmintis;
4. Įvedimo srautas.

- DT: Objekto, į kurį kopijuosime, numeris:

1. Vartotojo atmintis;
2. Supervizorinė atmintis;
3. Išvedimo srautas.

Kartu kanalų įrenginys turi komandą `XCHG`, tačiau neturi procesoriaus, kuris galėtų ją įvykdyti. `XCHG` komandą vykdo centrinis procesorius, todėl šis kanalų įrenginys nėra lygiagrečiai su centriniu procesoriumi veikianti aparatūra. Procesas, norėdamas pasinaudoti kanalų įrenginiu, turi nustatyti kanalų įrenginio registrus ir tada įvykdyti komandą `XCHG`.

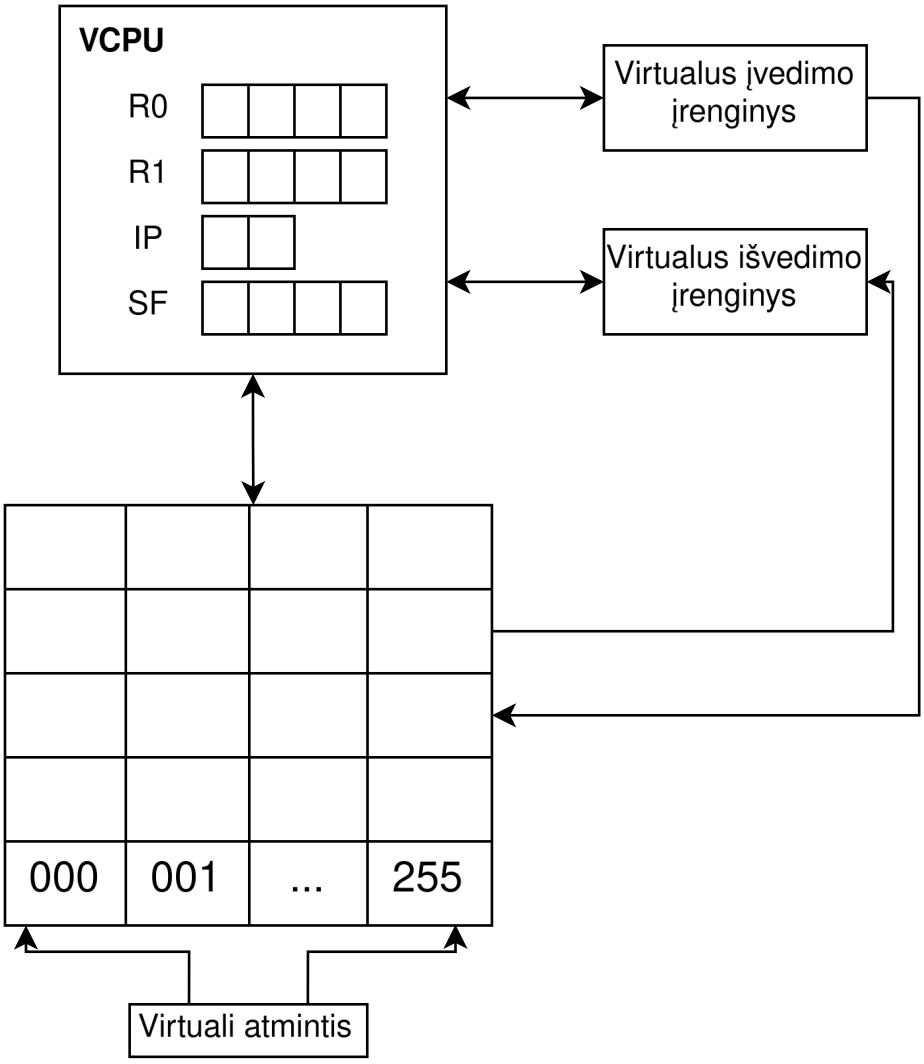
1.4. Įvedimo/išvedimo įrenginiai

Duomenims įvesti naudojama klaviatūra, o išvesti - ekranas.

2. Virtualios mašinos aprašas

2.1. Virtualios mašinos samprata

Virtuali mašina - tai tarsi virtuali realios mašinos kopija, kurioje surinktos reikalingos realios mašinos komponentės: procesorius, atmintis, įvedimo/išvedimo įrenginiai. Kiekvienai užduočiai operacinė sistema priskiria savo virtualią mašiną. Esminis virtualios mašinos privalumas yra tas, kad užduotis, kurią vykdo virtuali mašina, elgiasi lyg būtų vienintelė užduotis visoje mašinoje.



2 pav. Virtualios mašinos schema

2.2. Procesorius

Virtualios mašinos procesoriaus paskirtis - vykdyti programą, kuri yra virtualioje atmintyje.
Virtualaus procesoriaus registrai:

- `R0` – 4 baitų bendro naudojimo registras.

- `R1` – 4 baitų bendro naudojimo registras.
- `IP` - 2 baitų virtualios mašinos programos skaitliklis.
- `SF` – 4 baitų registras, kuriame po 1 baitą skirta keturioms žymėms: „zero flag“ (`ZF = 1`, kai operacijos rezultatas lygus nuliui, `ZF = 0` kitu atveju), „sign flag“ (`SF = 1`, kai operacijos rezultatas neigiamas, `SF = 0` kitu atveju), „interrupt flag“ (`IF = 1`, kai įvyksta pertraukimas iš išorinių įrenginių, `IF = 0` kitu atveju), „overflow flag“ (`OF = 1`, kai operacijos suma ar skirtumas išeina iš leistino intervalo rėžių ($[0000, FFFF]$ teigiamiem skaičiam ir $[-FFF, -001]$ neigiamiem skaičiam), `OF = 0` kitu atveju).

2.3. Procesoriaus komandos

Atmintyje esantis 4 baitų žodis gali būti traktuojamas kaip duomenys arba komanda. Jeigu komanda dirba su atmintimi, pirmieji du baitai (vyresnieji) laikomi operacijos kodu, likę naudojami kaip operandai (paprastai tai bus atminties adresas). Kitu atveju, visi 4 baitai gali būti laikomi operacijos kodu. Mikroprogramos, interpretuojančios virtualaus procesoriaus komandas bus vykdomos HLP. Pačios mikroprogramos turi būti tik skaitymui pažymėtoje supervizorinės atminties dalyje.

Virtualios mašinos procesoriaus komandos su paaiškinimais:

Jeigu x, y yra vartojami kaip adresai (dažniausiai x žymės atminties bloką, o y žodį tame bloke), tai $x, y \in \{0, 1, \ldots, F\}$.

- `LDxy`. Atminties ląstelės, kuri yra x bloke y žodyje, turinio kopijavimas į registrą `R0`, t.y. $R0 := [x \cdot 16 + y]$.
- `STxy`. Registro `R0` reikšmė įrašoma į atminties ląstelę,, kuri yra x bloke y žodyje, t.y. $[x \cdot 16 + y] := R0$.
- `SWAP`. Registrų `R0` ir `R1` reikšmės sukeičiamos vietomis, t.y. $[R0] \leftrightarrow [R1]$.
- `ADDX`. Registrų `R0` ir `R1` reikšmių suma patalpinama registre `R0`, t.y. $[R0] = [R0] + [R1]$. Pagal gautą rezultatą, nustatomi `ZF`, `OF` ir `SF`.
- `SUBX`. Registrų `R0` ir `R1` reikšmių skirtumas patalpinamas registre `R0`, t.y. $[R0] = [R0] - [R1]$. Pagal gautą rezultatą, nustatomi `ZF`, `OF` ir `SF`.
- `CMPX`. Registrų `R0` ir `R1` reikšmių palyginimas. Atliekama operacija $[R0] - [R1]$, kurios rezultatas neišsaugomas. Jei $[R0] = [R1]$, tai `ZF = 1`, `SF = 0`. Jei $[R0] < [R1]$, tai `ZF = 0`, `SF = 1`. Jei $[R0] > [R1]$, tai `ZF = 0`, `SF = 0`.
- `JPxy`. Valdymas perduodamas adresu $(x \cdot 16 + y)$.
- `JExy`. Jeigu `ZF = 1`, tai valdymas perduodamas adresu $(x \cdot 16 + y)$.
- `JNxy`. Jeigu `ZF = 0`, tai valdymas perduodamas adresu $(x \cdot 16 + y)$.
- `JGxy`. Jeigu `ZF = 0` ir `SF = OF`, tai valdymas perduodamas adresu $(x \cdot 16 + y)$.
- `JLxy`. Jeigu `ZF` $\neq$ `OF`, tai valdymas perduodamas adresu $(x \cdot 16 + y)$.
- `GETD`. Iš įvedimo srauto (klaviatūros) perskaito skaičių (iš leistino intervalo) dešimtainiu pavidalu ir įrašo jį į `R1` (šešioliktainiu pavidalu).

- `Pxyz`. Į išvedimo srautą išsiunčia tiek baitų, kiek nurodyta registre `R1`, pradedant atminties ląstele $(x \cdot 16 + y) \cdot 4 + z$, kur $x \in [0, 15]$ žymį atminties bloko numerį, $y \in [0, 15]$ - žodžio bloke numerį, o $z \in [0, 3]$ - baitų poslinkį žodyje.
- `GONx`. Paleidžiamas failas pavadinimu, kuris nurodytas registre `R1`, ir jai paduodamas atminties blokas x kaip parametras.
- `HALT` – Vartotojo programos vykdymo pabaiga.

Naudojantis šiomis komandomis ir bus rašomos užduočių programos, kurias vykdys virtualios mašinos.

2.4. Atmintis

Kiekvienai virtualiai mašinai operacinė sistema skiria 16 vartotojo atminties blokų, kuriuose patalpinta užduoties programa. Duomenų segmentui skiriami 4 blokai (t.y. žodžiai nuo 0 iki 63 imtinai), parametrams 1 blokas (t.y. žodžiai nuo 64 iki 79 imtinai), kodo segmentui 11 blokų (t.y. žodžiai nuo 80 iki 255 imtinai). Kiekvienas virtualios atminties blokas turi virtualų ir realų adresą. Virtualiais adresais operuoja virtuali mašina, realiais – reali mašina. Ryšiai tarp virtualaus ir realaus adreso nusakomi puslapių lentelėmis.

2.5. Puslapiavimas

Realios mašinos vartotojo atmintį sudaro 51 blokas. Kiekvienai naujai sukurtai virtualiąjai mašinai reikia skirti 17 blokų iš 51. Blokai gali būti parinkti bet kokiu būdu.

Puslapiavimo mechanizmo esmė: sakykime, kuriama nauja virtuali mašina, kuriai operacinė sistema suteikia 5 blokus vartotojo atminties, pvz., blokus numeriais 1, 3, 5, 7, 9. Pati virtuali mašina šiuos takelius mato sunumeruotus nuo 0 iki 4, t.y. realios atminties blokas nr. 1 virtualiai mašinai yra nr. 0, realios atminties blokas nr. 3 virtualiai mašinai yra nr. 1 ir t.t. Norint išlaikyti sąryšius tarp realių ir virtualių blokų adresų, naudojame puslapių lentelę, kuri sudaryta iš vieno bloko.

Virtualus bloko numeris	0	1	2	3	4
Realus bloko numeris	1	3	5	7	9

3 pav. Puslapių lentelės pavyzdys

Taigi, virtuali mašina, norėdama sužinoti realų bloko adresą, kreipiasi į savo puslapių lentelę ir nuskaito reikšmę, esančią žodyje su bloko numeriu, pvz., pirmo virtualios mašinos takelio realus numeris yra 3. Kadangi registro `PTR` reikšmė – puslapių lentelės adresas (puslapių lentelės

atminties bloko pirmo žodio adresas, t.y. adresas $xy0$, kur $xy \in \{00, 01, \ldots, 0F, \ldots, 43\}$), kiekviena virtuali mašina, prieš pradėdama darbą, nustatys šį registrą jai reikalinga reikšme. Virtualaus bloko x žodžio y realus adresas gaunamas pagal formulę:

$realusadresas = [PTR + x] \cdot 16 + y$

Norint gauti adresą baitų tikslumu, reikia padauginti formulę iš žodžio ilgio, t.y. 4.

Virtualios mašinos kūrimo ir veiklos scenarijus žvelgiant iš atminties pusės būtų toks:

1. Kuriant virtualią mašiną, operacinė sistema jai išskiria 16 blokų vartotojo atminties ir 1 bloką (paskutinį realios mašinos išskirtą) puslapių lentelei.
2. Puslapių lentelės adresas patalpinamas į `PTR` registrą, o pati puslapių lentelė (t.y. tas blokas) užpildoma išskirtų 16 blokų realiais numeriais.
3. Virtuali mašina baigiama kurti ir gauna procesorių.
4. Virtualiai mašinai prireikė paversti virtualų adresą, pvz., antrojo virtualios atminties bloko (t.y. nr. 1) šeštą žodį, kas būtų 16, realiu adresu.
5. Virtuali mašina, pasinaudojus duota formule, randa realų adresą (pagal duotą pvz., tai būtų 36).

2.6. Pertraukimai

Modelyje bus realizuoti trijų tipų pertraukimai – programiniai, supervizoriniai ir taimerio. Programinių pertraukimų registras yra `PI`, supervizorinių pertraukimų registras – `SI`, taimerio - `TI`. Programiniai pertraukimai kyla vykdant virtualios mašinos užduotį (programą), bandant įvykdyti kokį nors neleistiną veiksmą arba nuskaičius neleistiną reikšmę. Supervizoriniai pertraukimai kyla virtualiai mašinai norint įvykdyti veiksmą, kuris gali vykti tik supervizoriaus režime. Taimerio pertraukimai kyla, kai taimerio registro reikšmė pasiekia nulį (`TI = 0`). Pertraukimai gali būti aptikti tik vartotojo režime. Supervizoriniame režime centrinio procesoriaus darbo pertraukti negalima.

Pertraukimai gali kilti šiais būdais:

- Supervizorinius pertraukimus iššauks operacijos `GETD`, `Pxyz`, `HALT`, `GONx`: `GETD` nustato `SI = 1`, `Pxyz` $\Rightarrow$ `SI = 2`, `HALT` $\Rightarrow$ `SI = 3`, `GONx` $\Rightarrow$ `SI = 4`.
- Programiniai pertraukimai: `PI = 1` – neteisingas adresas, `PI = 2` – neteisingas operacijos kodas, `PI = 3` – neteisingas skaičius.
- Taimerio pertraukimai: esant `TI = 0`, fiksuojamas taimerio pertraukimas.

Esant situacijai `SI = 0` $\wedge$ `PI = 0` $\wedge$ `TI` $\neq$ `0` pertraukimų sistema neaptiks.

Pertraukimai nustatomi paprasčiausiai virtualaus procesoriaus registrams priskiriant atitinkamas reikšmes (pavyzdžiui, komandų interpretatoriui vykdant komandą `GETD`, jis priskiria `SI = 1`). Kiekvieną kartą komandų interpretatoriui įvykdžius komandą, kviečiama funkcija `test()`, kuri apklausia registrus, ir, jei kilo pertraukimas, gražina informaciją apie tai.

Funkcija `test()` gali būti tokia:

```
if ((PI + SI) > 0) or (TI = 0) then return 1 else return 0;
```

Nustačius įvykusį pertraukimą (t.y. `test()` $\neq$ `0`) virtualios mašinos procesoriaus registrų reikšmės yra išsaugomos, procesorius perjungiamas į supervizoriaus režimą, nustatomas pertraukimo pobūdis (pavyzdžiui, tiesiogiai apklausiant registrus) ir kviečiama pertraukimą apdorosianti paprogramė. Vėliau, kaip buvo minėta, valdymas grįžta į virtualią mašiną, registrai atstatomi, procesorius perjungiamas į vartotojo režimą, ir operacinė sistema sprendžia, ką daryti toliau.

2.7. Įvedimas ir išvedimas

Duomenims įvesti naudojama klaviatūra, o išvesti - ekranas. Naudodamasi komandomis `GETD` ir `Pxyz` bei jų sukeliamais pertraukimais virtuali mašina inicijuoja įvestį ir išvestį.

2.8. Vykdomieji failai

Programos (užduotys) bus laikomos failuose, su kuriais dirbs aukšto lygio kalbos procesorius. Failai kietajame diske saugomi nuosekliai, iš failo viršaus ir apačios atskiriant juos keturiais minuso (`----`) simboliais. Po atskiriamojo ženklo, pirmi keturi baitai nurodo failo (ne programos) pavadinimą. Norint sukurti naują programą, reikia sukurti ir teisingai užpildyti naują tekstinį failą modelio išorėje. Programa susideda iš šių dalių:

- Pradžios žymė
- Programa
- Pabaigos žymė

2.8.1. Pradžios žymė

Programą pradeda pradžios žymė „$RJG“, kuriai skirti 4 baitai (t.y. vienas žodis).

2.8.2. Programa

Prieš kiekvieną naują programos ar duomenų žodį, turi būti nurodyta virtualios mašinos atminties ląstelė, į kurią talpinti po šios komandos einantį žodį, pvz.:

```
$50$
ADDX
SUBX
$65$
CMPX
```

Taip aprašyta programa nurodo į virtualios mašinos atminties ląstelę adresu `50` patalpinti komandą `ADDX`, adresu `51` patalpinti komandą `SUBX`, o adresu `65` patalpinti komandą `CMPX`.

2.8.3. Pabaigos žymė

Programą užbaigia pabaigos žymė „$END“, kuriai skirti 4 baitai (t.y. vienas žodis).